

DSA STUDY BLUEPRINT SERIES

89. Morris Inorder Traversal

Threaded Binary Tree Traversals (Morris Traversal)

Section 09 • C++ Core Data Structures & Algorithms

Core Concepts & Visual Blueprint

Theoretical models and memory architectures

Key Principles

- **Concept:** Inorder traversal with $O(1)$ space.
- Build temporary threads from predecessor back to current node.

Memory & Execution Blueprint

Left subtree's right-most leaf node temporarily points to root [Thread Link]

C++ Implementation Reference

Standard code layout and syntax

```
void morrisInorder(TreeNode* root) {  
    TreeNode* curr = root;  
    while (curr) {  
        if (!curr->left) { cout << curr->val << " "; curr = curr->right; }  
        else {  
            TreeNode* pred = curr->left;  
            while (pred->right && pred->right != curr) pred = pred->right;  
            if (!pred->right) { pred->right = curr; curr = curr->left; }  
            else { pred->right = nullptr; cout << curr->val << " "; curr = curr->right; }  
        }  
    }  
}
```

Algorithmic Complexity & Best Practices

Performance evaluations and critical guidelines

Performance Table

Aspect / Case	Complexity
Time Complexity	$O(N)$ (Revisits edges at most 3 times)
Space Complexity	$O(1)$ (No recursion stack)

Key Practices & Gotchas

- **Important:** Cut temporary threads during traversal to restore the tree's original structure.
- Verify boundary conditions (e.g. empty inputs, single elements).
- Optimize dynamic memory allocations to prevent leaks.

Dry Run & Step-by-Step Execution

Trace analysis on a sample input case

Execution Walkthrough

Let's trace recursive Preorder Traversal visits (Node-Left-Right) for tree `1 (left=2, right=3)`:

Call Stack Level	Active Node Visited	Expected Left Traversal	Expected Right Traversal
1	Root (1)	Recurse on Left child (2)	Recurse on Right child (3)
2	Leaf (2)	Base case (nullptr)	Base case (nullptr)

Interview Variations & Optimization Pathways

Common follow-up problems and optimization strategies

Key Variations & Follow-up Questions

- **Iterative traversals:** Standard DFS traversals can run iteratively by simulating recursion calls with Stacks.
- **Level Order BFS:** Level tracking searches use Queues to visit node layers.
- **Related LeetCode Problems:** #102 Binary Tree Level Order Traversal, #94 Binary Tree Inorder Traversal, #236 Lowest Common Ancestor.